



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Jesús Alexis Pérez León

N° de Cuenta: 314207850

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 16-abril-2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1. Agregar luz de tipo spotlight para el helicóptero de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight en el suelo hacia adelante y al retroceder (mover con teclado hacia dirección de X positiva) ilumine con un spotlight el suelo hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo a la dirección en la que esté moviéndose el helicóptero.
- 2.- Crear o instanciar una pecera (traslúcida) con las normales hacia adentro y en la parte superior imagen de agua e instanciar dentro de la pecera al pez abisal con movimiento de teclado para subir y bajar en diagonal.
- 3.- Agregar una luz de tipo puntual de color azul ligada al bulbo del pez que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara de su práctica 7 como esta luz (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual), de tal forma que se pueda ver: las 2 luces apagadas, las 2 luces prendidas, una luz prendida y una luz apagada y viceversa.
4. Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del bulbo del pez y que ilumine en cualquier dirección dentro de la pecera al presionar teclas asignadas al eje x, eje y y eje z.

- 1.-Para encender una luz delantera cuando el helicóptero avanza hacia adelante y una luz trasera cuando retrocede, primero implemente dos spotLights sobre el helicóptero.

```
//Luz amarilla para el helicoptero trasera
spotLights[1] = SpotLight(1.0f, 1.0f, 0.0f,
    1.0f, 0.5f,
    0.0f, 5.0f, 16.0f,
    0.0f, 0.0f, -1.0f,
    1.0f, 0.0f, 0.0f,
    20.0f);
spotLightCount++;
```

```
//Luz amarilla para el helicoptero frontal
spotLights[3] = SpotLight(1.0f, 1.0f, 0.0f,
    1.0f, 0.5f,
    0.0f, 5.0f, 16.0f,
    0.0f, 1.0f, -1.0f,
    1.0f, 0.0f, 0.0f,
    20.0f);
spotLightCount++;
```

Después usando dos setFlash para cada luz del helicóptero y con la jerarquía del mismo las luces (spotlights) se movían junto con el helicóptero.

```
//setFlash de la luz amartilla frontal para que semueva junto con el helicoptero
glm::vec3 heliPos = glm::vec3(modelauxHelicoptero * glm::vec4(0.0f, 3.0f, 0.0f, 1.0f));
glm::vec3 heliDir = glm::normalize(glm::vec3(modelauxHelicoptero * glm::vec4(0.0f, 1.0f, -1.0f, 0.0f)));
spotLights[3].setFlash(heliPos, heliDir);

//setFlash de la luz amartilla trasera para que semueva junto con el helicoptero
glm::vec3 heliPos2 = glm::vec3(modelauxHelicoptero * glm::vec4(0.0f, -5.0f, 0.0f, 1.0f));
glm::vec3 heliDir2 = glm::normalize(glm::vec3(modelauxHelicoptero * glm::vec4(0.0f, -1.0f, -1.0f, 0.0f)));
spotLights[1].setFlash(heliPos2, heliDir2);
```

Para encender la luz correspondiente al movimiento del helicóptero implemente dos variables booleanas sobre la definición de acción de cada tecla donde las vriables son opuestas y esto indica al hsader que debe o no dibujar la spotlight en función.

```
//mueve hacia atras el helicoptero y enciende la luz amarilla trasera
if (key == GLFW_KEY_H)
{
    theWindow->muevexHelicoptero += 1.0;
    theWindow->luzFrontHelicoptero = false;
    theWindow->luzBackHelicoptero = true;
}

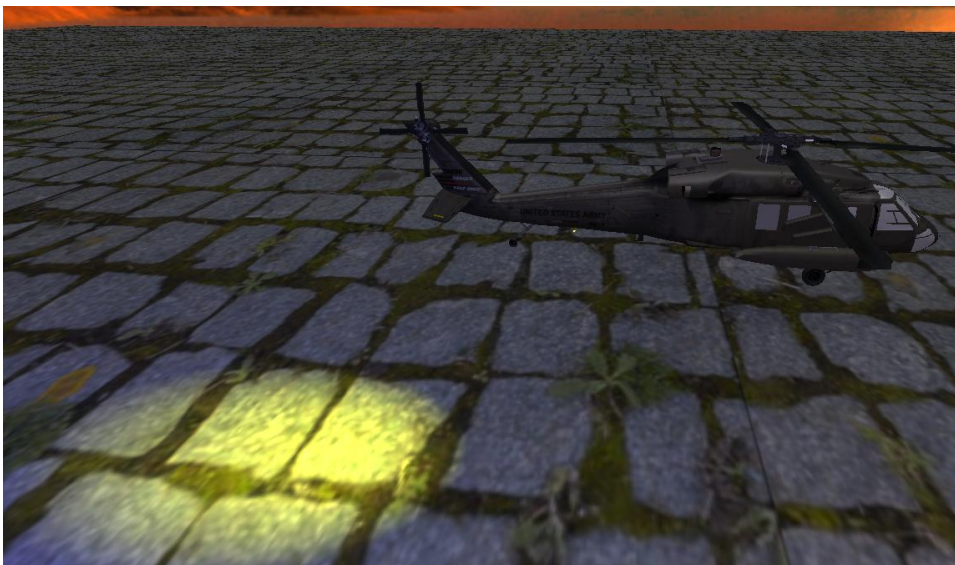
//mueve hacia adelante el helicoptero y enciende la luz amarilla frontal
if (key == GLFW_KEY_J)
{
    theWindow->muevexHelicoptero -= 1.0;
    theWindow->luzBackHelicoptero = false;
    theWindow->luzFrontHelicoptero = true;
}
```

```
int lucesActivas = 0;

if (mainWindow.getLuzFrontHelicoptero()) {
    // CASO: AVANCE FRONTAL (Encender Frontal [3], Apagar Trasera [1])
    // Intercambiamos la 1 con la 3 para que la Frontal esté en una posición visible (índice 1)
    std::swap(spotLights[1], spotLights[3]);

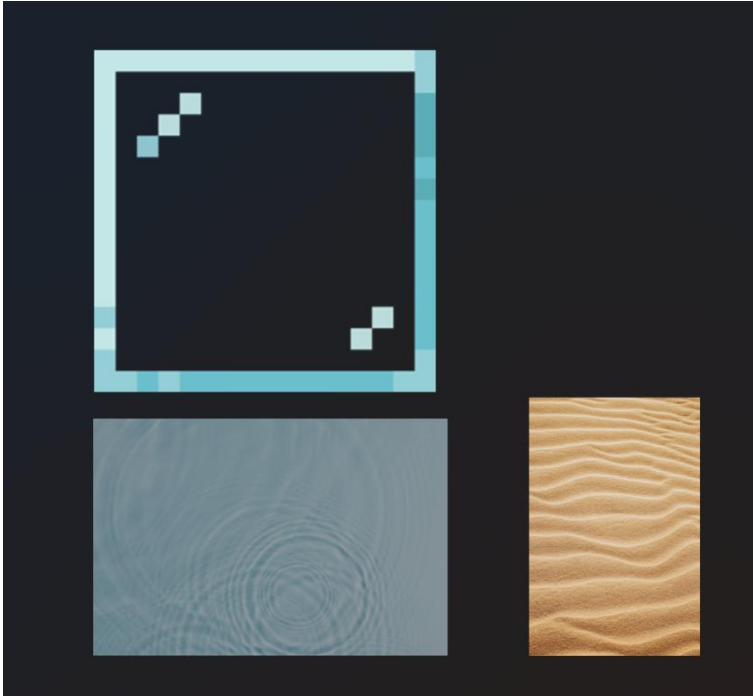
    // Enviamos 4 luces, pero como la trasera ahora está en el índice 3
    // y solo queremos 3 luces totales (Flash, Frontal, Auto), mandamos 3:
    shaderList[0].SetSpotLights(spotLights, 3);

    // IMPORTANTE: Regresar al orden original para que SetFlash no se confunda el siguiente frame
    std::swap(spotLights[1], spotLights[3]);
}
else if (mainWindow.getLuzBackHelicoptero()) {
    // CASO: AVANCE TRASERO (Encender Trasera [1], Apagar Frontal [3])
    // Aquí no hay que swapear nada porque la Trasera ya está en el índice 1.
    // Simplemente enviamos 3 luces. El shader ignorará la luz 3 (Frontal).
    shaderList[0].SetSpotLights(spotLights, 3);
}
else {
    // CASO: ESTÁTICO (Ambas apagadas)
    // Movemos la 1 y la 3 al final del arreglo y enviamos solo las luces 0 y 2
    // Para simplificar, si no se mueve, enviamos solo la luz 0 (linterna)
    shaderList[0].SetSpotLights(spotLights, 1);
}
```



2.- Para la pecera, me apoye de la función que dibuja un dado visto en practicas anteriores la modifique para que generara un prisma rectangular (forma de pecera). Esto genera un cuerpo rectangular solido simple.

Para las texturas genere una imagen PNG (para usar transparencia) y en ella reuni 3 texturas (vidrio, arena y agua) obtenidas de google fotos.



Especifique las zonas de las texturas en cada cara de triangulo y agregue las normales hacia a dentro de la pecera.

```
void CrearPecera()  
{  
    unsigned int indices[] = {  
        // En una pecera, queremos ver el interior, por lo que  
        // invertimos el orden de los índices (Clockwise) para que  
        // el culling no oculte las caras internas.  
  
        // frontal (z = 0.5)  
        0, 2, 1,  
        2, 0, 3,  
  
        // trasera (z = -0.5)  
        8, 10, 9,  
        10, 8, 11,  
  
        // izquierda (x = -1.0)  
        12, 14, 13,  
        14, 12, 15,  
  
        // derecha (x = 1.0)  
        4, 6, 5,  
        6, 4, 7,  
  
        // abajo (y = -0.5)  
        16, 18, 17,  
        18, 16, 19,  
  
        // arriba (y = 0.5)  
        20, 22, 21,  
        22, 20, 23  
    };  
};
```

```

GLfloat vertices[] = {
    // x      y      z      S      T      NX      NY      NZ

    // Frontal (Normal hacia adentro: +Z -> -Z)
    -1.0f, -0.5f,  0.5f,  0.0f, 0.44f,  0.0f,  0.0f,  1.0f, // 0
     1.0f, -0.5f,  0.5f,  0.56f, 0.44f,  0.0f,  0.0f,  1.0f, // 1
     1.0f,  0.5f,  0.5f,  0.56f, 1.0f,  0.0f,  0.0f,  1.0f, // 2
    -1.0f,  0.5f,  0.5f,  0.0f, 1.0f,  0.0f,  0.0f,  1.0f, // 3

    // Derecha (Normal hacia adentro: +X -> -X)
     1.0f, -0.5f,  0.5f,  0.0f, 0.44f,  1.0f,  0.0f,  0.0f, // 4
     1.0f, -0.5f, -0.5f,  0.56f, 0.44f,  1.0f,  0.0f,  0.0f, // 5
     1.0f,  0.5f, -0.5f,  0.56f, 1.0f,  1.0f,  0.0f,  0.0f, // 6
     1.0f,  0.5f,  0.5f,  0.0f, 1.0f,  1.0f,  0.0f,  0.0f, // 7

    // Trasera (Normal hacia adentro: -Z -> +Z)
    -1.0f, -0.5f, -0.5f,  0.0f, 0.44f,  0.0f,  0.0f, -1.0f, // 8
     1.0f, -0.5f, -0.5f,  0.56f, 0.44f,  0.0f,  0.0f, -1.0f, // 9
     1.0f,  0.5f, -0.5f,  0.56f, 1.0f,  0.0f,  0.0f, -1.0f, // 10
    -1.0f,  0.5f, -0.5f,  0.0f, 1.0f,  0.0f,  0.0f, -1.0f, // 11

    // Izquierda (Normal hacia adentro: -X -> +X)
    -1.0f, -0.5f, -0.5f,  0.0f, 0.44f, -1.0f,  0.0f,  0.0f, // 12
    -1.0f, -0.5f,  0.5f,  0.56f, 0.44f, -1.0f,  0.0f,  0.0f, // 13
    -1.0f,  0.5f,  0.5f,  0.56f, 1.0f, -1.0f,  0.0f,  0.0f, // 14
    -1.0f,  0.5f, -0.5f,  0.0f, 1.0f, -1.0f,  0.0f,  0.0f, // 15

    // Abajo (Normal hacia adentro: -Y -> +Y)
    -1.0f, -0.5f,  0.5f,  0.72f, 0.0f,  0.0f, -1.0f,  0.0f, // 16
     1.0f, -0.5f,  0.5f,  1.0f, 0.0f,  0.0f, -1.0f,  0.0f, // 17
     1.0f, -0.5f, -0.5f,  1.0f, 0.42f,  0.0f, -1.0f,  0.0f, // 18
    -1.0f, -0.5f, -0.5f,  0.72f, 0.42f,  0.0f, -1.0f,  0.0f, // 19

    // Arriba (Normal hacia adentro: +Y -> -Y)
    -1.0f,  0.5f,  0.5f,  0.0f, 0.0f,  0.0f,  1.0f,  0.0f, // 20
     1.0f,  0.5f,  0.5f,  0.58f, 0.0f,  0.0f,  1.0f,  0.0f, // 21
     1.0f,  0.5f, -0.5f,  0.58f, 0.39f,  0.0f,  1.0f,  0.0f, // 22
    -1.0f,  0.5f, -0.5f,  0.0f, 0.39f,  0.0f,  1.0f,  0.0f, // 23
};

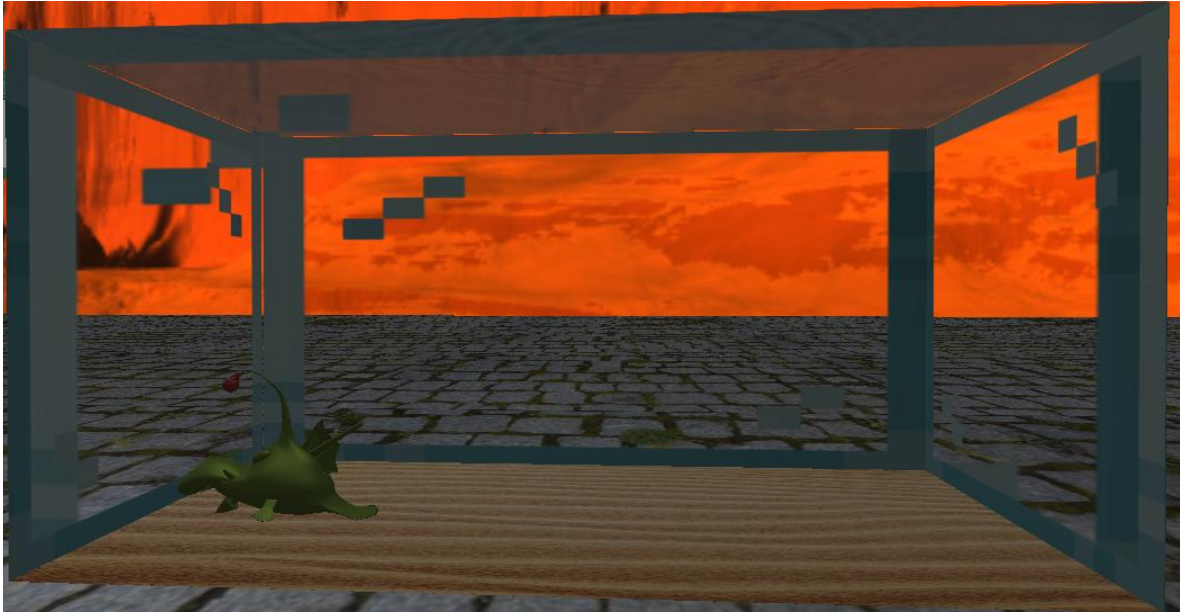
Mesh* pecera = new Mesh();
// 24 vértices * 8 componentes cada uno = 192 floats
// 12 triángulos * 3 índices = 36 índices
pecera->CreateMesh(vertices, indices, 192, 36);
meshList.push_back(pecera);
}

```

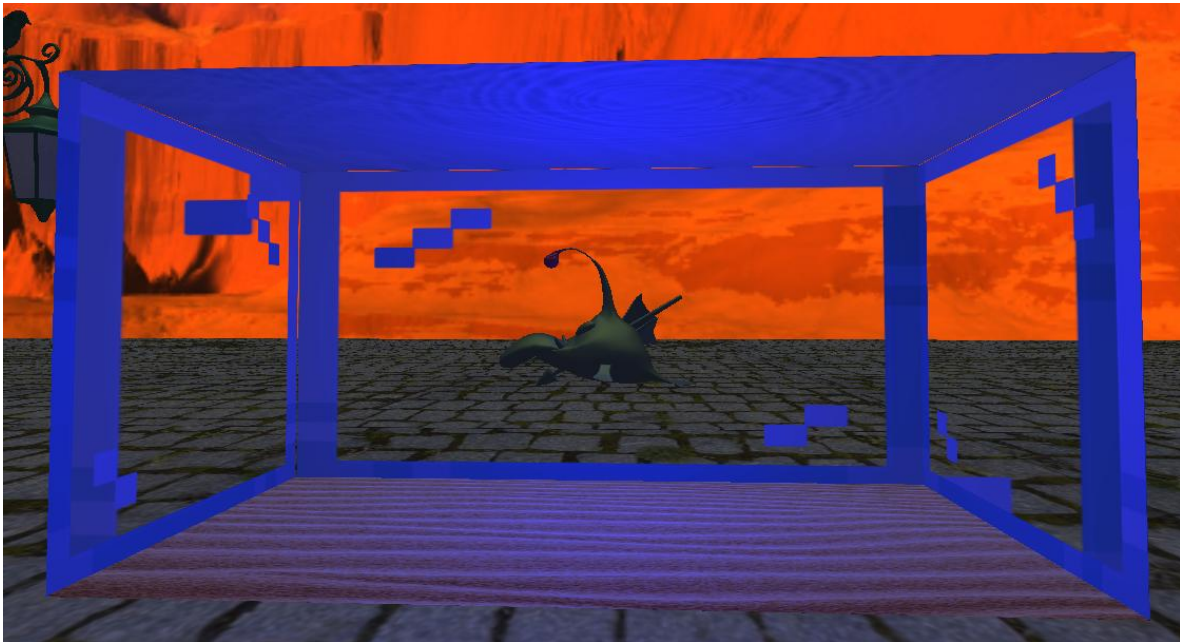
Es importante el orden de dibujo de cada cara ya que en OpenGL importa ya que se implementa una jerarquía de dibujo y en función a lo deseado debemos dar prioridad a ciertas caras.

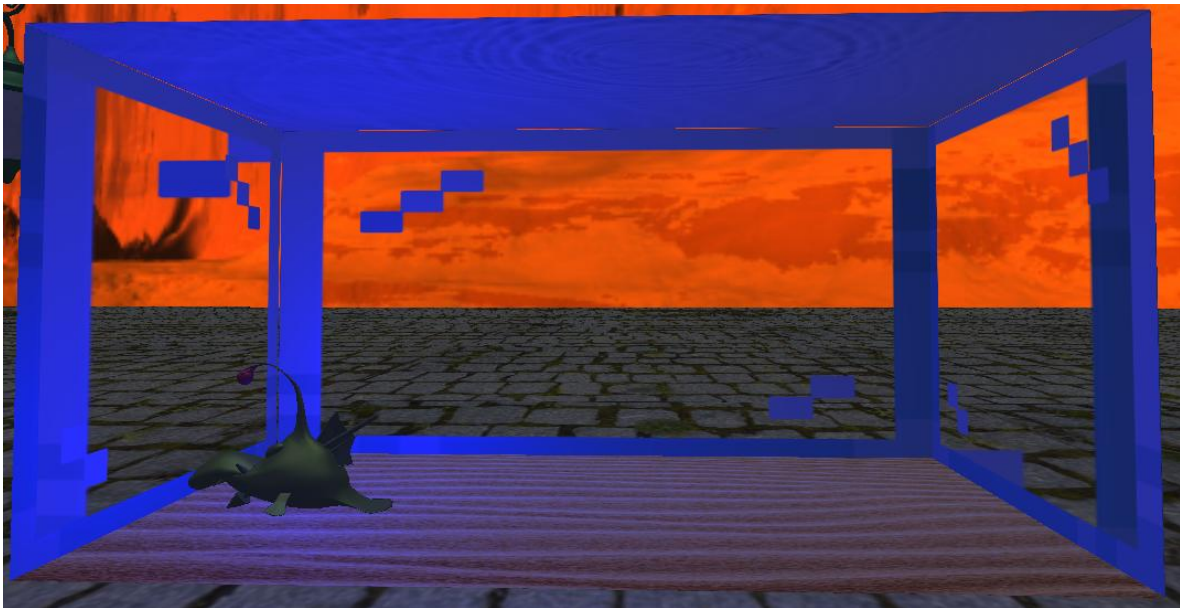
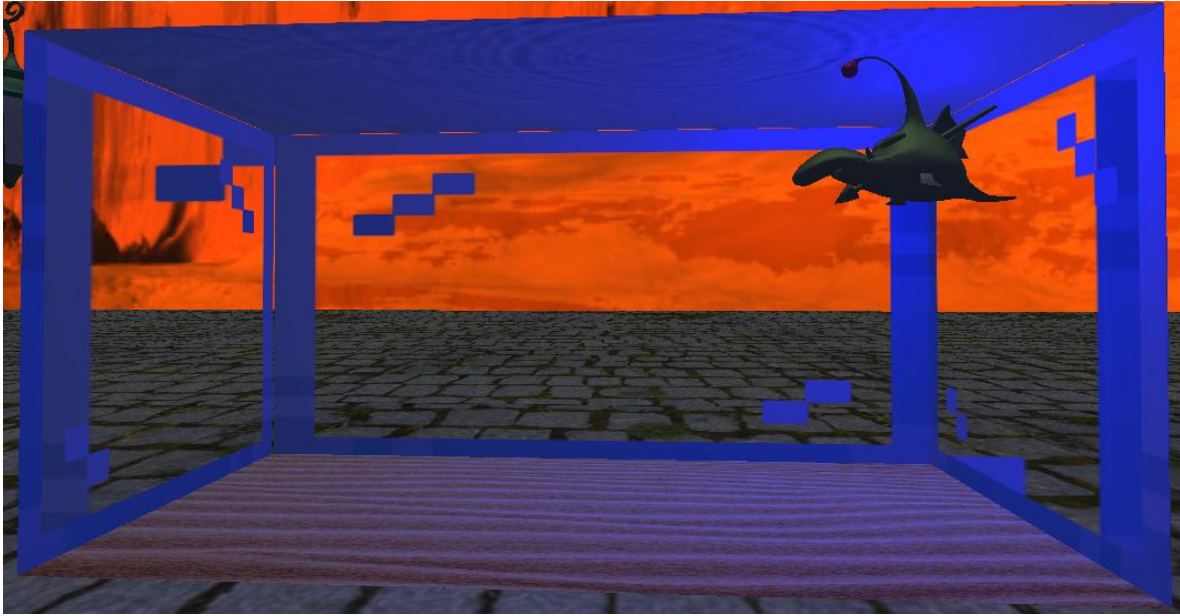
Agregue el pez_abisal en tres partes y con jerarquía cada una desde el cuerpo. Los modelos proporcionados por el profesor lo genere en medio, le di unos colores con UniformColor. Y por ultimo asigne dos teclas "R" y "F" para que el pez se traslade en forma de diagonal en la pecera. Este movimiento se basa en manipular las componentes XY del Translate y limitarlo para que no salga de la pecera.





3.- Agregue una pointLight color azul acomodada en el bulbo del pez. Para que esta se moviera junto con el pez, agregue una función de setPosition en PointLigth.cpp y PointLight.cpp, y así mediante un setFlash la luz puntual se mueve junto con el pez en diagonal.





Para apagar y prender la luz del bulbo del pez ahora usamos corrimientos de las luces ya que además del pez la lámpara publica de la practica pasada debe también ser independiente así que se implementa un arreglo auxiliar que hace intercambios según la función de la tecla y se manda o no el shader en guncion a la función llamada.

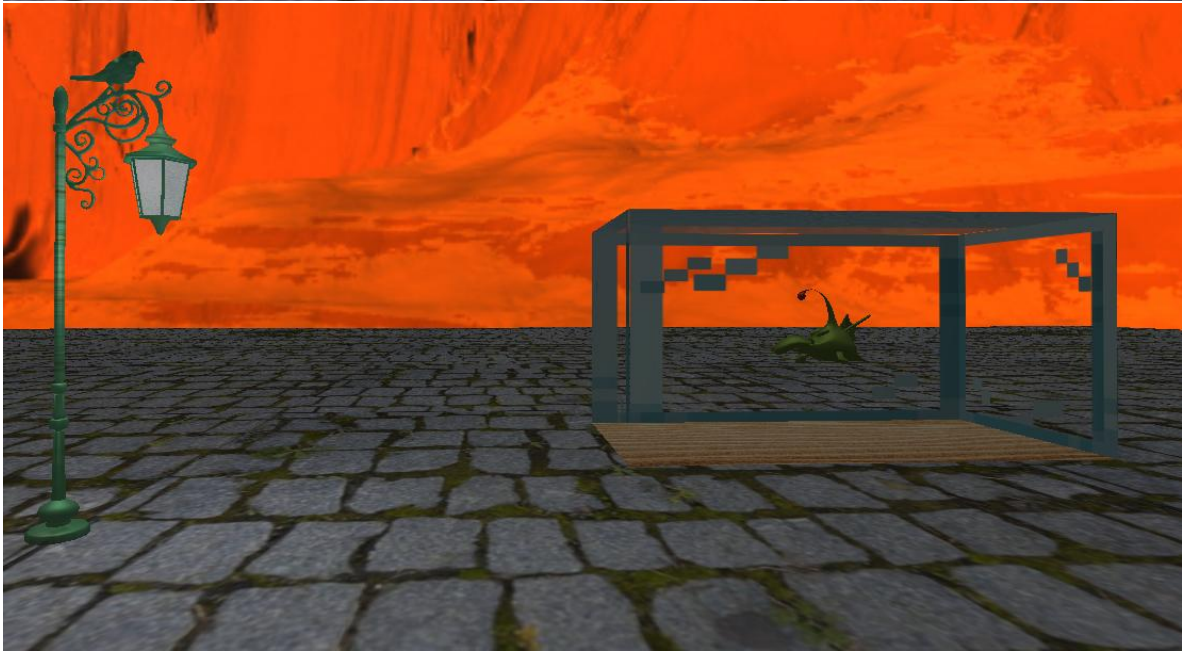
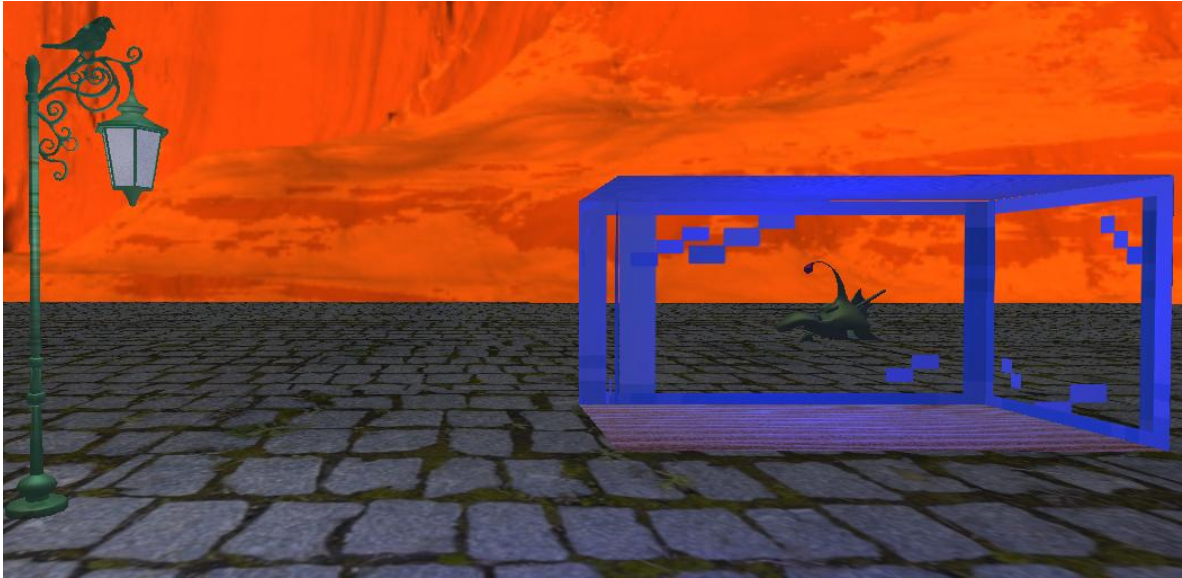
```
//////////
// Lógica para enviar solo las luces puntuales activas al shader tanto para la farola como para el bulbo del pez.
//arreglo temporal para enviar al shader
PointLight lucesPuntualesActivas[3];
int contadorActivas = 0;

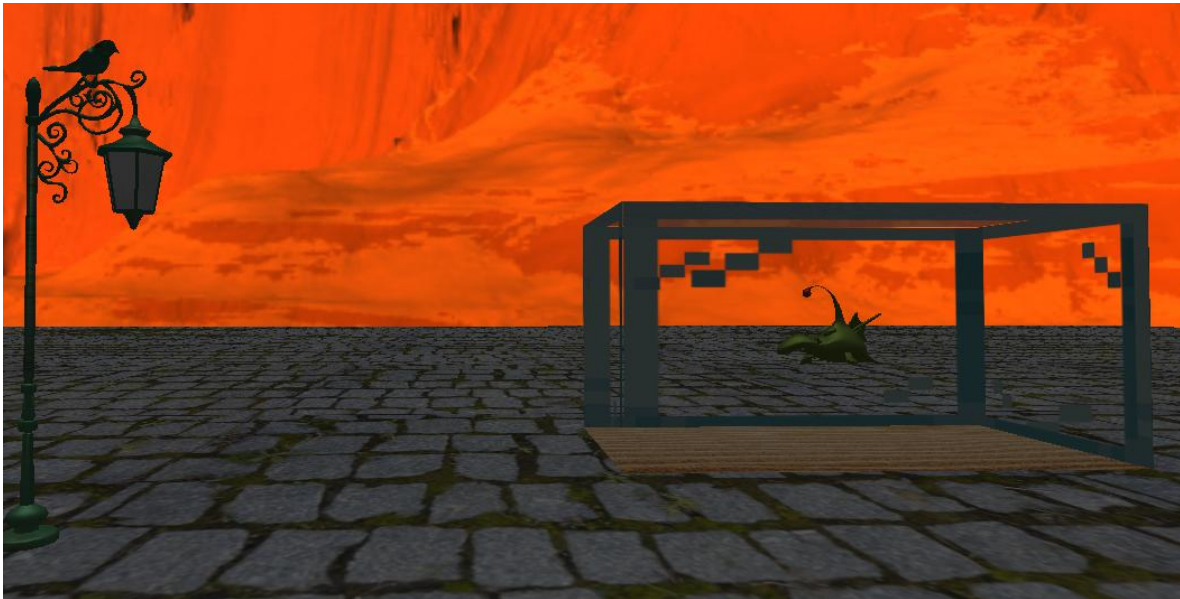
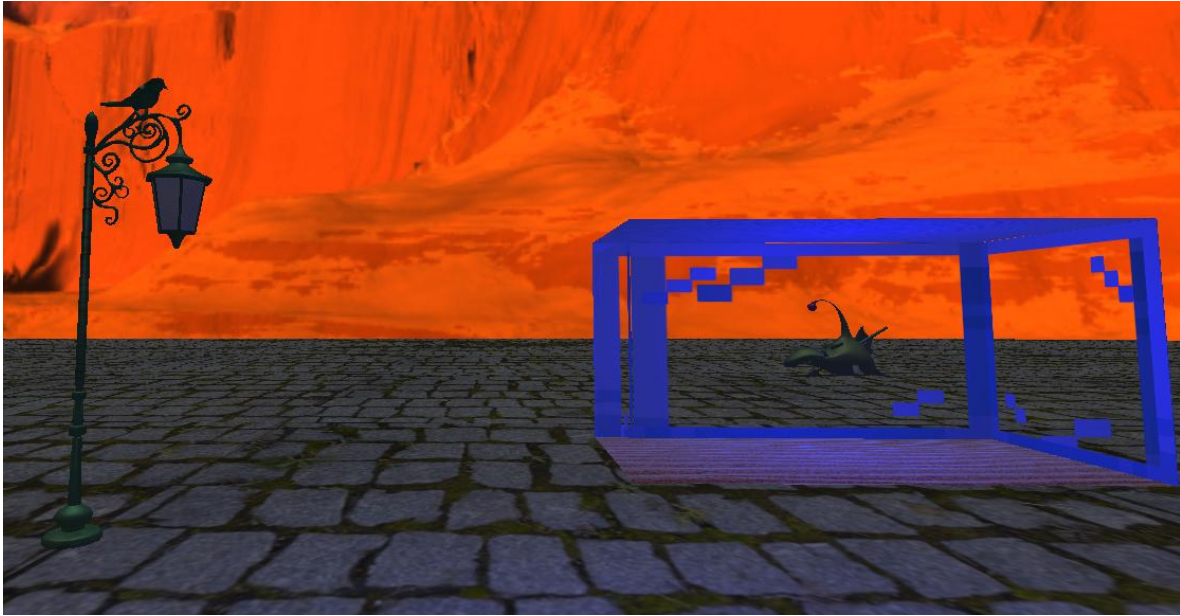
//agregamos la luz [0] (luz roja) es parte de practicas anteriores y siempre está activa, por lo que no necesita evaluación
lucesPuntualesActivas[contadorActivas] = pointLights[0];
contadorActivas++;

// evakua Luz [1] (Lámpara Pública)
if (mainWindow.getLuzPrendida()) {
    lucesPuntualesActivas[contadorActivas] = pointLights[1];
    contadorActivas++;
}

// Evaluar Luz [2] (Bulbo Pez)
if (mainWindow.getBulboPrendido()) {
    lucesPuntualesActivas[contadorActivas] = pointLights[2];
    contadorActivas++;
}

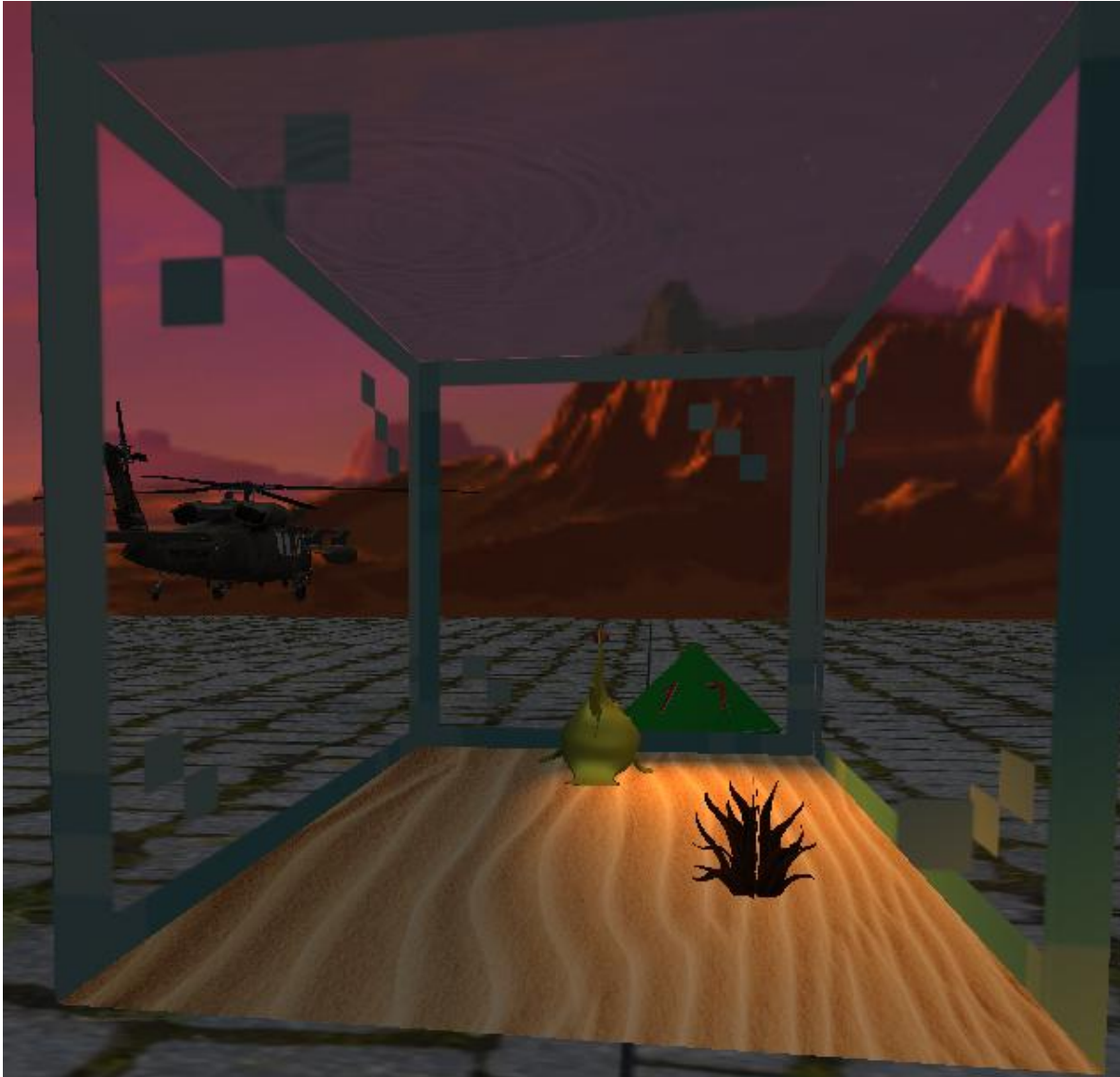
//Enviar solo las que están activas
// El shader solo hará el ciclo 'for' hasta 'contadorLucesActivas'
shaderList[0].SetPointLights(lucesPuntualesActivas, contadorActivas);
```





4.- para la spotlight del bulbo del pez tridireccional . y la asocie por jerarquía al bulbo del pez. Despues mediante tres teclas que aumentan valores cada que se presiona para Especifique una tecla (flecha, izquierda, fleta arriba y flecha derecha) y en ellas se manejan valores entre -1 y 1. Y asi en los vlaores del vec3 se especifica cada función y esto aparenta apuntar de manera libre.

```
// spotlight (direccional) del bulbo del pez que sigue la jerarquía del pez
glm::vec3 spotLightPezPos = glm::vec3(modelPez * glm::vec4(0.0f, 0.0f, 1.0f, 1.0f)); // acomodo del spotlight bulbo pez (offset)
glm::vec3 spotLightPezDir = glm::normalize(glm::vec3(modelauxPez * glm::vec4(mainWindow.getDirx(), 0.0f + mainWindow.getDiry(), 0.0f +
spotLights[0].setFlash(spotLightPezPos, spotLightPezDir);
```



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Me costó tiempo entender cómo apagar y prender luces en función a cómo mandarlas a la jugada.

3.- Conclusión:

En esta practica es importante rescatar que en OpenGL es importante el orden en la que seceden las cosas. Ya que el cuidado de esos detalles resultan en escenas mas realistas.